WEB TASARIMI VE PROGRAMLAMA

LAB-4

Tailwind CSS · Üretken UI · Component Yaklaşımı

İçindekiler

1 LAB-4 Öğrenme Hedefleri **3**

2 Utility-First CSS Nedir? **3**
2.1 Neden Utility-First? 4

3 Tailwind CSS Kurulumu ve Yapılandırma **5**

4 Temel Utility Class'lar **7**

5 Responsive Class'lar **9**

6 Dark Mode **11**

7 Component Yaklaşımı: Neden ve Nasıl? **12**

8 Button Component Varyantları **14**

9 Input, Card ve Alert Component'leri **15**
9.1 Input Component 15
9.2 Card Component 17
9.3 Alert Component 18

10 Uygulama: Portföyü Tailwind'e Taşı **20**

11 UI Kit Sayfası Oluşturma **23**

12 Git İş Akışı ve Teslim **26**

13 Teslim Kontrol Listesi ve Değerlendirme Rubriği **27**
13.1 Değerlendirme Rubriği 28

14 Kendini Test Et **28**

15 Ek Kaynaklar ve İleri Okuma **29**

1 LAB-4 Öğrenme Hedefleri

Önceki laboratuvarlarda geliştirme ortamını kurmuş (LAB-1), semantik ve erişilebilir bir HTML iskeleti oluşturmuş (LAB-2) ve bu iskeleti Flexbox/Grid ile responsive hale getirmiştin (LAB-3). CSS değişkenleri, fluid typography ve mobile-first breakpoint'lerle design token altyapısını da kurmuştun.

Şimdi sıra **CSS yazma hızını dramatik şekilde artırmaya** geldi. Bu laboratuvarda **Tailwind CSS** framework'ünü öğrenecek, **utility-first** yaklaşımla hızlıca UI oluşturacak ve kendi **component kütüphaneni** tasarlayacaksın.

Bu laboratuvarın sonunda aşağıdaki becerilere sahip olacaksın:

1. **Utility-first CSS mantığını anlamak:** Geleneksel CSS yazımı ile utility-first yaklaşım arasındaki farkı kavramak ve Tailwind'in felsefesini benimsemek.
2. **Tailwind CSS'i projeye kurmak ve yapılandırmak:** Vite + React projesine Tailwind v4 entegrasyonu yapmak.
3. **Responsive class'lar kullanmak:** Tailwind'in `sm:`, `md:`, `lg:` prefix'leri ile mobile-first responsive tasarım yapmak.
4. **Dark mode desteği eklemek:** `dark:` prefix'i ile karanlık tema varyantlarını tanımlamak.
5. **Component yaklaşımıyla UI parçaları üretmek:** Button, Input, Card ve Alert gibi temel bileşenleri birden fazla varyantla oluşturmak.
6. **Portföy sayfasını Tailwind'e taşımak:** LAB-2/3'te saf CSS ile yazdığın portföyü Tailwind class'larına dönüştürmek (veya yeni bir landing page üretmek).
7. **UI Kit sayfası oluşturmak:** En az 8 component varyantını tek bir sayfada sergileyen bir UI Kit hazırlamak.

Benzetme ile Anlama

LAB-3'te CSS'i "sıfırdan tuğla dökerek" yazdık — her renk, her boşluk, her boyut elle tanımlandı. Tailwind CSS ise **hazır tuğla seti** gibidir: Önceden kesilmiş, standart boyutlarda, etiketlenmiş tuğlalar. Sen sadece doğru tuğlayı seçip yerine koyarsın. Daha hızlı, daha tutarlı ve daha az hata yaparsın — ama tuğlaların nasıl üretildiğini bilmen gerekir (ki LAB-3'te öğrendin).

Sırada Ne Var?

Hedeflerimiz belirlendi. İlk adım olarak "utility-first CSS" kavramını tanıyalım ve Tailwind'in neden bu kadar popüler olduğunu anlayalım.

2 Utility-First CSS Nedir?

Geleneksel CSS'te bir butonu stillemek için önce bir class adı icat eder, sonra o class'a kurallar yazarsın. Utility-first yaklaşımda ise her CSS özelliğine karşılık gelen **küçük, tek amaçlı class'lar** doğrudan HTML'de kullanılır.

Teorik Arka Plan

Geleneksel (Semantic CSS) yaklaşım:

```html
<button class="btn-primary">Gonder</button>
```

```css
/* ayri bir CSS dosyasinda */
.btn-primary {
  background-color: #1E3A8A;
  color: white;
  padding: 0.5rem 1.5rem;
  border-radius: 8px;
  font-weight: 600;
  border: none;
  cursor: pointer;
  transition: background-color 150ms ease;
}
.btn-primary:hover {
  background-color: #2563EB;
}
```

Utility-first (Tailwind) yaklaşım:

```html
<button class="bg-blue-800 text-white px-6 py-2
               rounded-lg font-semibold border-0
               cursor-pointer transition-colors
               hover:bg-blue-600">
  Gonder
</button>
```

İkinci örnekte ayrı bir CSS dosyası yazmana gerek yok. Her class tek bir şey yapar: `bg-blue-800` arka plan rengini, `px-6` yatay padding'i, `rounded-lg` border-radius'u belirler.

2.1 Neden Utility-First?

Teorik Arka Plan

Avantajları:

- **Hız:** CSS dosyası ile HTML arasında gidip gelmezsin. Stil değişikliği doğrudan HTML'de yapılır.
- **İsimlendirme sorunu yok:** `.card-wrapper-inner-container` gibi saçma class isimleri icat etmene gerek kalmaz.
- **Ölü kod yok:** Kullanılmayan CSS kuralları birikmez. Tailwind yalnızca kullandığın class'ları üretir (purge/tree-shaking).
- **Tutarlılık:** Herkes aynı spacing, renk ve boyut skalasını kullanır. "Bu padding 14px mı 16px mı?" tartışması biter.
- **Responsive ve state varyantları:** `md:flex-row`, `hover:bg-blue-600`, `dark:bg-gray-900` gibi prefix'lerle CSS yazmadan responsive, hover ve dark mode desteği eklersin.

Eleştiriler ve yanıtları:

- *"HTML çok kalabalık oluyor"* → Doğru, ama modern IDE'ler class'ları katlar ve component yaklaşımıyla tekrar azaltılır.
- *"Inline style'dan ne farkı var?"* → Tailwind class'ları responsive prefix, hover/focus state, dark mode, design token tutarlılığı sağlar. Inline style bunların hiçbirini yapamaz.
- *"CSS öğrenmeden geçilemez"* → Kesinlikle! LAB-3'te saf CSS'i öğrendin. Tailwind, CSS bilginin üzerine inşa edilen bir **verimlilik aracıdır**, CSS'in yerine geçmez.

Gerçek Hayatta Karşılığı

Bir aşçı düşün. Önce malzemeleri tek tek tanıması, bıçak tekniklerini öğrenmesi gerekir (LAB-3 = saf CSS). Sonra mutfak robotu kullanmaya başlar — doğrama, karıştırma, hamur yoğurma işleri çok hızlanır. Ama malzemeyi tanımayan biri mutfak robotuyla iyi yemek yapamaz. Tailwind da CSS bilen geliştiriciler için bir "mutfak robotu"dur.

Sırada Ne Var?

Utility-first felsefesini anladık. Şimdi Tailwind CSS'i projemize kuralım ve yapılandıralım.

3 Tailwind CSS Kurulumu ve Yapılandırma

Tailwind CSS v4 (2024 sonu itibarıyla en güncel sürüm), Vite ile entegre çalışacak şekilde tasarlanmıştır. Kurulum oldukça basittir.

Uygulama Adımı

Uygulama-1: Tailwind CSS v4 Kurulumu

1. Mevcut Vite + React projenin klasöründe olduğundan emin ol (LAB-1'de oluşturduğun proje veya yeni bir proje):

```
# Yeni proje olusturmak istersen:
npm create vite@latest lab4-tailwind -- --template react
cd lab4-tailwind
npm install
```

2. Tailwind CSS v4'ü kur:

```
npm install tailwindcss @tailwindcss/vite
```

3. `vite.config.js` dosyasını güncelle — Tailwind plugin'i ekle:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    react(),
```

```
    tailwindcss(),
  ],
})
```

4. Ana CSS dosyanı (`src/index.css`) güncelle — en üste Tailwind import'unu ekle:

```
@import "tailwindcss";
```

5. Projeyi çalıştır ve test et:

```
npm run dev
```

Uygulama Adımı

Uygulama-2: Tailwind Yapılandırması — Özel Tema

Tailwind v4'te tema özelleştirmesi doğrudan CSS dosyasında `@theme` direktifi ile yapılır. LAB-3'teki design token'larını Tailwind'e taşıyalım:

```
@import "tailwindcss";

@theme {
  --color-primary: #1E3A8A;
  --color-secondary: #2563EB;
  --color-accent: #7C3AED;
  --color-surface: #F9FAFB;
  --color-muted: #6B7280;
  --color-error: #DC2626;
  --color-success: #16A34A;

  --font-family-sans: system-ui, -apple-system,
                      sans-serif;
  --font-family-mono: 'Fira Code', 'Consolas',
                      monospace;
}
```

Bu tanımlamadan sonra `bg-primary`, `text-secondary`, `font-sans` gibi class'lar otomatik olarak kullanılabilir hale gelir.

Dikkat – Sık Yapılan Hatalar

- Tailwind v4, v3'ten farklı bir yapılandırma sistemine sahiptir. Eski `tailwind.config.js` dosyası v4'te **artık gerekmez**; her şey CSS içinde `@theme` ile yapılır.
- `@import "tailwindcss"` satırı CSS dosyasının **en üstünde** olmalıdır. Altına kendi özel kurallarını yazabilirsin.
- Tailwind class'ları çalışmıyorsa: (1) `npm run dev`'i yeniden başlat, (2) `@import "tailwindcss"` satırının mevcut olduğunu kontrol et, (3) Vite config'inde plugin'in eklendiğinden emin ol.

Derinlemesine Anlama

Tailwind Nasıl Çalışır? (Perde Arkası)
Tailwind bir **build-time** aracıdır. Projeni derlerken (veya dev sunucusu çalışırken):

1. Tüm kaynak dosyalarını (HTML, JSX, TSX) tarar.
2. Kullanılan class isimlerini tespit eder (örn. `bg-blue-800`, `md:flex-row`).
3. Yalnızca bu class'lar için CSS kuralları üretir.
4. Kullanılmayan class'lar **hiç üretilmez** — bu sayede final CSS dosyası çok küçük olur (genelde 10–30 KB).

Bu, LAB-3'te yazdığın saf CSS'ten farklıdır: Orada tüm kurallar her zaman yüklenir. Tailwind'de yalnızca ihtiyacın olan kurallar var olur.

Sırada Ne Var?

Kurulum tamam, tema özelleştirmesini yaptık. Şimdi Tailwind'in en temel utility class'larını öğrenelim — spacing, renk, tipografi, border ve gölge.

4 Temel Utility Class'lar

Tailwind'de her CSS özelliğine karşılık gelen bir class ailesi vardır. Aşağıda en sık kullanılan kategorileri ve class isimlerini öğreneceksin.

Teorik Arka Plan

1. Spacing (Boşluk): `p-*` (padding), `m-*` (margin), `gap-*`
Tailwind'in varsayılan spacing skalası 4px çarpanlarına dayanır:

Class	Değer	Piksel
`p-0`	0	0px
`p-1`	0.25rem	4px
`p-2`	0.5rem	8px
`p-4`	1rem	16px
`p-6`	1.5rem	24px
`p-8`	2rem	32px
`p-12`	3rem	48px
`p-16`	4rem	64px

Yön belirtme: `px-*` (sol+sağ), `py-*` (üst+alt), `pt-*` (üst), `pb-*` (alt), `pl-*` (sol), `pr-*` (sağ). Margin için aynı mantık: `mx-*`, `my-*`, `mt-*`, vb.
2. Renk: `bg-*` (arka plan), `text-*` (metin), `border-*` (kenarlık)
Tailwind'in renk paleti 50–950 tonlarından oluşur:
`bg-blue-50` (çok açık) → `bg-blue-500` (orta) → `bg-blue-950` (çok koyu)
Yaygın renkler: `slate`, `gray`, `zinc`, `red`, `orange`, `amber`, `yellow`, `lime`, `green`, `emerald`, `teal`, `cyan`, `sky`, `blue`, `indigo`, `violet`, `purple`, `fuchsia`, `pink`, `rose`.
3. Tipografi:

- Font boyutu: `text-sm`, `text-base`, `text-lg`, `text-xl`, `text-2xl`, `text-3xl`, `text-4xl`
- Font ağırlığı: `font-light`, `font-normal`, `font-medium`, `font-semibold`, `font-bold`

- Satır yüksekliği: `leading-tight`, `leading-normal`, `leading-relaxed`
- Hizalama: `text-left`, `text-center`, `text-right`

4. Border ve Radius:

- Kenarlık: `border`, `border-2`, `border-4` (kalınlık)
- Kenarlık rengi: `border-gray-300`, `border-blue-500`
- Radius: `rounded-sm`, `rounded`, `rounded-md`, `rounded-lg`, `rounded-xl`, `rounded-full`

5. Gölge: `shadow-sm`, `shadow`, `shadow-md`, `shadow-lg`, `shadow-xl`

6. Layout:

- Display: `flex`, `grid`, `block`, `inline-block`, `hidden`
- Flex: `flex-row`, `flex-col`, `flex-wrap`, `items-center`, `justify-between`, `gap-4`
- Grid: `grid-cols-2`, `grid-cols-3`, `grid-cols-4`, `col-span-2`
- Boyut: `w-full`, `w-1/2`, `w-auto`, `h-screen`, `max-w-4xl`

Uygulama Adımı

Uygulama-3: İlk Tailwind Bileşeni

`App.jsx` dosyasını açıp mevcut içeriği sil ve aşağıdaki basit kartı yaz:

```
<div class="min-h-screen bg-gray-50 flex
            items-center justify-center p-4">
  <div class="bg-white rounded-xl shadow-lg
              max-w-sm w-full p-6">
    <h1 class="text-2xl font-bold text-gray-900
               mb-2">
      Merhaba Tailwind!
    </h1>
    <p class="text-gray-600 mb-4">
      Bu benim ilk Tailwind CSS şbileenim.
      Her class tek bir is yapar.
    </p>
    <button class="bg-blue-600 text-white px-4
                   py-2 rounded-lg font-medium
                   hover:bg-blue-700
                   transition-colors">
      Devam Et
    </button>
  </div>
</div>
```

Tarayıcıda açıp sonucu gör. Hiçbir CSS dosyası yazmadın ama stillenmiş bir kart elde ettin!

Dikkat – Sık Yapılan Hatalar

- Tailwind class'ları **ezberlenmez**, zamanla **kas hafızasına** yerleşir. İlk hafta sürekli dokümantasyona bakman normal. https://tailwindcss.com/docs en iyi referans.

- VS Code kullanıyorsan **Tailwind CSS IntelliSense** eklentisini mutlaka kur. Class isimlerini otomatik tamamlar ve hover'da CSS çıktısını gösterir.
- Tailwind class sıralaması önemlidir (okunabilirlik için). Önerilen sıra: layout → boyut → spacing → tipografi → renk → border → efekt → state. **Prettier plugin for Tailwind** bu sıralamayı otomatik yapar.

Sırada Ne Var?

Temel class'ları öğrendik. Şimdi Tailwind'in en güçlü özelliklerinden birine geçelim: **responsive prefix'ler**.

5 Responsive Class'lar

LAB-3'te media query'ler yazarak responsive tasarım yapmıştın. Tailwind'de aynı şeyi **prefix'ler** ile doğrudan HTML'de yaparsın — ayrı media query yazmana gerek kalmaz.

Teorik Arka Plan

Tailwind'in responsive prefix'leri **mobile-first** çalışır (LAB-3'teki `min-width` mantığı):

Prefix	Min-width	CSS Karşılığı
(yok)	0px	Varsayılan (mobil)
`sm:`	640px	`@media (min-width: 640px)`
`md:`	768px	`@media (min-width: 768px)`
`lg:`	1024px	`@media (min-width: 1024px)`
`xl:`	1280px	`@media (min-width: 1280px)`
`2xl:`	1536px	`@media (min-width: 1536px)`

Kullanım:

```html
<!-- Mobilde dikey, tablette yatay duzen -->
<div class="flex flex-col md:flex-row gap-4">

  <!-- Mobilde tam genislik, masaustunde 1/3 -->
  <aside class="w-full lg:w-1/3 bg-gray-100 p-4">
    Sidebar
  </aside>

  <!-- Geri kalan alan -->
  <main class="flex-1 p-4">
    Ana icerik
  </main>
</div>
```

Bu kod şu anlama gelir:

- **Mobil (0–767px):** `flex-col` → elemanlar dikey yığılır, `w-full` → sidebar tam genişlik.
- **Tablet (768px+):** `md:flex-row` → elemanlar yatay dizilir.
- **Masaüstü (1024px+):** `lg:w-1/3` → sidebar genişliğin 1/3'ünü alır.

Uygulama Adımı

Uygulama-4: Responsive Grid
Proje kartlarını responsive bir grid içinde göster:

```
<section class="px-4 py-12 max-w-6xl mx-auto">
  <h2 class="text-3xl font-bold text-center mb-8">
    Projelerim
  </h2>

  <!-- Mobil:1 sutun, Tablet:2 sutun, Desktop:3 -->
  <div class="grid grid-cols-1 sm:grid-cols-2
              lg:grid-cols-3 gap-6">

    <article class="bg-white rounded-lg shadow-md
                    overflow-hidden">
      <img src="proje1.jpg" alt="E-Ticaret"
           class="w-full h-48 object-cover" />
      <div class="p-4">
        <h3 class="font-semibold text-lg mb-2">
          E-Ticaret Sitesi</h3>
        <p class="text-gray-600 text-sm">
          React ve Node.js ile.</p>
      </div>
    </article>

    <!-- Diger kartlar ayni yapiyla... -->
  </div>
</section>
```

LAB-3'te aynı düzeni Grid + media query ile yapmıştın. Tailwind'de `grid-cols-1 sm:grid-cols-2 lg:grid-cols-3` ile 3 kelimede halloldu!

Derinlemesine Anlama

Responsive Prefix Mantığı
Prefix'siz class **tüm ekranlarda** geçerlidir. Prefix'li class yalnızca o breakpoint'ten **itibaren** geçerlidir ve daha küçük ekranı **ezmez**.
Örnek: `text-sm md:text-base lg:text-lg`

- 0–767px: `text-sm` geçerli.
- 768–1023px: `md:text-base` devreye girer, `text-sm`'yi ezer.
- 1024px+: `lg:text-lg` devreye girer, `md:text-base`'i ezer.

Bu tam olarak LAB-3'teki mobile-first `min-width` mantığıdır — sadece sözdizimi farklı.

Sırada Ne Var?

Responsive class'ları öğrendik. Şimdi kullanıcı deneyimini bir adım ileri taşıyacak özelliğe geçelim: **dark mode**.

6 Dark Mode

Karanlık tema (dark mode) artık bir lüks değil, kullanıcı beklentisidir. Tailwind'de dark mode desteği `dark:` prefix'i ile sağlanır.

Teorik Arka Plan

Tailwind Dark Mode Stratejileri:

1. **Sistem tercihi (varsayılan):** Kullanıcının işletim sistemi temasına göre otomatik geçiş. CSS `prefers-color-scheme` media query'si kullanılır.
2. **Manuel geçiş (selector):** HTML'de `<html class="dark">` ekleyerek JavaScript ile kontrol edilir. Kullanıcıya tema değiştirme butonu sunulur.

Kullanım:

```
<div class="bg-white dark:bg-gray-900
            text-gray-900 dark:text-gray-100">

  <h1 class="text-blue-800 dark:text-blue-300">
    Baslik
  </h1>

  <p class="text-gray-600 dark:text-gray-400">
    Bu metin karanlik temada acik gri olur.
  </p>

  <button class="bg-blue-600 dark:bg-blue-500
                 text-white hover:bg-blue-700
                 dark:hover:bg-blue-400">
    Buton
  </button>
</div>
```

Her eleman için hem açık hem karanlık tema renklerini tanımla. `dark:` prefix'i yalnızca karanlık tema aktifken devreye girer.

Uygulama Adımı

Uygulama-5: Dark Mode Toggle Ekle

1. Tailwind v4'te selector stratejisi için CSS'e ekle:

```
@import "tailwindcss";

@custom-variant dark (&:where(.dark, .dark *));
```

2. Basit bir tema değiştirme butonu oluştur:

```
<button
  onclick="document.documentElement
           .classList.toggle('dark')"
  class="fixed top-4 right-4 z-50
         bg-gray-200 dark:bg-gray-700
         text-gray-800 dark:text-gray-200
         p-2 rounded-full shadow-lg
```

```
          hover:scale-110 transition-transform"
  aria-label="Tema degistir">
  <!-- Acik temada ay, karanlik temada gunes -->
  <span class="dark:hidden">&#9790;</span>
  <span class="hidden dark:inline">&#9728;</span>
</button>
```

3. Sayfadaki tüm bileşenlere `dark:` varyantlarını ekle: arka plan, metin rengi, kenarlık, gölge.

Dikkat – Sık Yapılan Hatalar

- Dark mode eklerken **sadece arka planı** karartmak yetmez. Metin rengi, kenarlık rengi, gölge, input arka planı, placeholder rengi — hepsinin dark varyantı olmalıdır.
- Renk kontrastını her iki temada da kontrol et. Açık temada okunaklı olan metin, karanlık temada kaybolabilir (veya tam tersi).
- `dark:` prefix'i `hover:`, `focus:` gibi prefix'lerle birleştirilebilir: `dark:hover:bg-blue-400`. Sıra önemlidir.

Sırada Ne Var?

Dark mode ile temamızı zenginleştirdik. Şimdi en önemli konuya geçelim: **Component yaklaşımı**. Aynı buton stilini 15 yere kopyalamak yerine nasıl tek bir bileşen oluştururuz?

7 Component Yaklaşımı: Neden ve Nasıl?

Utility-first CSS'in bir dezavantajı, aynı class kombinasyonunu birden fazla yerde tekrar etmektir. Aynı buton stilini 10 farklı yerde kullandığını düşün — biri değiştiğinde 10 yeri güncellemen gerekir. **Component yaklaşımı** bu sorunu çözer.

Teorik Arka Plan

Component yaklaşımında iki strateji vardır:
1. React Component'i ile soyutlama (önerilen):
Tekrar eden class'ları bir React bileşenine sararız:

```
// components/Button.jsx
export default function Button({
  children,
  variant = "primary",
  size = "md",
  ...props
}) {
  const base = `inline-flex items-center
    justify-center font-medium rounded-lg
    transition-colors focus:outline-none
    focus:ring-2 focus:ring-offset-2`;
```

```

  const variants = {
    primary: `bg-blue-600 text-white
      hover:bg-blue-700
      focus:ring-blue-500
      dark:bg-blue-500 dark:hover:bg-blue-400`,
    secondary: `bg-gray-200 text-gray-800
      hover:bg-gray-300
      focus:ring-gray-500
      dark:bg-gray-700 dark:text-gray-200
      dark:hover:bg-gray-600`,
    danger: `bg-red-600 text-white
      hover:bg-red-700
      focus:ring-red-500
      dark:bg-red-500 dark:hover:bg-red-400`,
    ghost: `bg-transparent text-gray-700
      hover:bg-gray-100
      focus:ring-gray-400
      dark:text-gray-300
      dark:hover:bg-gray-800`,
  };

  const sizes = {
    sm: "px-3 py-1.5 text-sm",
    md: "px-4 py-2 text-base",
    lg: "px-6 py-3 text-lg",
  };

  return (
    <button
      className={`${base} ${variants[variant]}
                  ${sizes[size]}`}
      {...props}
    >
      {children}
    </button>
  );
}
```

Kullanımı:

```
<Button variant="primary" size="lg">Kaydet</Button>
<Button variant="secondary">Iptal</Button>
<Button variant="danger" size="sm">Sil</Button>
<Button variant="ghost">Detay</Button>
```

2. CSS'te `@apply` ile soyutlama (dikkatli kullan):

```
@layer components {
  .btn-primary {
    @apply bg-blue-600 text-white px-4 py-2
           rounded-lg font-medium
```

```
            hover:bg-blue-700 transition-colors;
  }
}
```

`@apply`, Tailwind class'larını geleneksel CSS class'ına dönüştürür. Ancak Tailwind ekibi bunu **nadiren** kullanmayı önerir — React component yaklaşımı daha güçlü ve esnektir.

Derinlemesine Anlama

Component Varyantları Neden Önemli?
İyi bir component kütüphanesi **varyant** (variant) sistemi sunar:

- **Renk varyantı:** primary, secondary, danger, success, warning, ghost
- **Boyut varyantı:** sm, md, lg
- **Durum varyantı:** disabled, loading
- **Şekil varyantı:** solid (dolgulu), outline (çerçeveli), ghost (şeffaf)

Bu varyant kombinasyonları "component varyantı" sayılır. Örneğin:

- Primary + Medium + Solid = 1 varyant
- Danger + Small + Outline = 1 varyant
- Ghost + Large + Disabled = 1 varyant

Sırada Ne Var?

Component yaklaşımının felsefesini anladık. Şimdi Button'dan başlayarak 4 temel bileşenimizi oluşturacağız.

8 Button Component Varyantları

Button, en temel ve en çok kullanılan UI bileşenidir. İyi bir Button component'i birden fazla renk, boyut ve stil varyantını desteklemelidir.

Uygulama Adımı

Uygulama-6: Button Component — 4 Renk × 3 Boyut
`src/components/Button.jsx` dosyası oluştur ve Bölüm 7'deki kodu uygula. Ardından UI Kit sayfasında tüm varyantları sergile:

```
<section class="space-y-8 p-8">
  <h2 class="text-2xl font-bold">Button</h2>

  <!-- Boyut varyantlari -->
  <div class="flex flex-wrap items-end gap-4">
    <Button size="sm">Kucuk</Button>
    <Button size="md">Orta</Button>
    <Button size="lg">Buyuk</Button>
  </div>

  <!-- Renk varyantlari -->
  <div class="flex flex-wrap gap-4">
```

```
    <Button variant="primary">Primary</Button>
    <Button variant="secondary">Secondary</Button>
    <Button variant="danger">Danger</Button>
    <Button variant="ghost">Ghost</Button>
  </div>

  <!-- Disabled durumu -->
  <div class="flex flex-wrap gap-4">
    <Button variant="primary" disabled
      class="opacity-50 cursor-not-allowed">
      Disabled
    </Button>
  </div>
</section>
```

Dikkat – Sık Yapılan Hatalar

- Button'da `focus:ring-*` class'ını **asla** kaldırma. Bu, klavye kullanıcılarının butonun focus'ta olduğunu görmesini sağlar (LAB-2'deki erişilebilirlik kuralı).
- `disabled` durumda butona `opacity-50 cursor-not-allowed` ekle ve `pointer-events-none` ile tıklamayı engelle.
- Buton içinde ikon kullanıyorsan, ikonun erişilebilirliğini sağla: `aria-hidden="true"` (dekoratif ikon) veya `aria-label` (tek başına anlam taşıyan ikon buton).

Sırada Ne Var?

Button component'i tamam. Şimdi diğer üç temel bileşeni oluşturalım: Input, Card ve Alert.

9 Input, Card ve Alert Component'leri

Her UI kütüphanesinin olmazsa olmaz bileşenlerinden üçünü daha oluşturacağız. Her biri birden fazla varyanta sahip olacak.

9.1 Input Component

Uygulama Adımı

Uygulama-7: Input Component
`src/components/Input.jsx` dosyası oluştur:

```
export default function Input({
  label,
  type = "text",
  error,
  helpText,
  id,
```

```
  ...props
}) {
  return (
    <div className="space-y-1">
      {label && (
        <label htmlFor={id}
          className="block text-sm font-medium
            text-gray-700
            dark:text-gray-300">
          {label}
        </label>
      )}
      <input
        id={id}
        type={type}
        className={`w-full px-3 py-2 rounded-lg
          border transition-colors
          focus:outline-none focus:ring-2
          dark:bg-gray-800 dark:text-gray-100
          ${error
            ? "border-red-500 focus:ring-red-500"
            : "border-gray-300 focus:ring-blue-500"
          }
          ${props.disabled
            ? "bg-gray-100 cursor-not-allowed"
            : "bg-white dark:bg-gray-800"
          }`}
        aria-describedby={
          error ? `${id}-error`
          : helpText ? `${id}-help`
          : undefined
        }
        {...props}
      />
      {error && (
        <p id={`${id}-error`} role="alert"
          className="text-sm text-red-600
                     dark:text-red-400">
          {error}
        </p>
      )}
      {helpText && !error && (
        <p id={`${id}-help`}
          className="text-sm text-gray-500
                     dark:text-gray-400">
          {helpText}
        </p>
      )}
    </div>
  );
```

```
}
```

Varyantları:

```
<Input id="name" label="Ad Soyad"
       placeholder="Ahmet Yilmaz" />
<Input id="email" label="E-posta" type="email"
       helpText="Ornek: ad@mail.com" />
<Input id="pass" label="Sifre" type="password"
       error="En az 8 karakter olmali" />
<Input id="disabled" label="Devre Disi"
       disabled value="Duzenlenemez" />
```

9.2 Card Component

Uygulama Adımı

Uygulama-8: Card Component

`src/components/Card.jsx` dosyası oluştur:

```
export default function Card({
  title,
  children,
  image,
  imageAlt,
  footer,
  variant = "elevated",
}) {
  const variants = {
    elevated: `bg-white dark:bg-gray-800
      shadow-md hover:shadow-lg`,
    outlined: `bg-white dark:bg-gray-800
      border border-gray-200
      dark:border-gray-700`,
    filled: `bg-gray-100 dark:bg-gray-800`,
  };

  return (
    <div className={`rounded-xl overflow-hidden
      transition-shadow ${variants[variant]}`}>
      {image && (
        <img src={image} alt={imageAlt || ""}
          className="w-full h-48 object-cover" />
      )}
      <div className="p-5">
        {title && (
          <h3 className="text-lg font-semibold
            text-gray-900 dark:text-gray-100
            mb-2">
            {title}
```

```
          </h3>
        )}
        <div className="text-gray-600
          dark:text-gray-400">
          {children}
        </div>
      </div>
      {footer && (
        <div className="px-5 py-3 bg-gray-50
          dark:bg-gray-900 border-t
          border-gray-100 dark:border-gray-700">
          {footer}
        </div>
      )}
    </div>
  );
}
```

Varyantları:

```
<Card variant="elevated" title="Proje A"
      image="proje.jpg"
      imageAlt="Proje ekran goruntusu">
  <p>Bu bir elevated (golgelii) kart.</p>
</Card>
<Card variant="outlined" title="Proje B">
  <p>Bu bir outlined (cerceveli) kart.</p>
</Card>
<Card variant="filled" title="Proje C"
      footer={<Button size="sm">Detay</Button>}>
  <p>Bu bir filled (dolgulu) kart.</p>
</Card>
```

9.3 Alert Component

Uygulama Adımı

Uygulama-9: Alert Component

`src/components/Alert.jsx` dosyası oluştur:

```
export default function Alert({
  children,
  variant = "info",
  title,
  dismissible = false,
  onDismiss,
}) {
  const variants = {
    info: `bg-blue-50 border-blue-500
      text-blue-800 dark:bg-blue-950
```

```
      dark:text-blue-200`,
    success: `bg-green-50 border-green-500
      text-green-800 dark:bg-green-950
      dark:text-green-200`,
    warning: `bg-amber-50 border-amber-500
      text-amber-800 dark:bg-amber-950
      dark:text-amber-200`,
    error: `bg-red-50 border-red-500
      text-red-800 dark:bg-red-950
      dark:text-red-200`,
  };

  return (
    <div role="alert"
      className={`border-l-4 rounded-r-lg p-4
        ${variants[variant]}`}>
      <div className="flex justify-between
                      items-start">
        <div>
          {title && (
            <p className="font-semibold mb-1">
              {title}
            </p>
          )}
          <p className="text-sm">{children}</p>
        </div>
        {dismissible && (
          <button onClick={onDismiss}
            className="ml-4 opacity-60
              hover:opacity-100"
            aria-label="Kapat">
            &#10005;
          </button>
        )}
      </div>
    </div>
  );
}
```

Varyantları:

```
<Alert variant="info" title="Bilgi">
  Formunuz basariyla kaydedildi.
</Alert>
<Alert variant="success" title="Basarili">
  Islem tamamlandi!
</Alert>
<Alert variant="warning" title="Uyari">
  Oturum 5 dakika sonra sona erecek.
</Alert>
<Alert variant="error" title="Hata" dismissible
```

```
        onDismiss={() => console.log('kapatildi')}>
  Baglanti kurulamadi. Tekrar deneyin.
</Alert>
```

Dikkat – Sık Yapılan Hatalar

- Alert bileşeninde `role="alert"` eklemeyi unutma — ekran okuyucu bu bölgeyi anında sesli okur (LAB-2'den hatırlayın).
- Dismissible (kapatılabilir) Alert'te kapat butonuna `aria-label="Kapat"` ekle; içinde sadece "X" ikonu varsa ekran okuyucu ne olduğunu bilemez.
- Dark mode renkleri eklerken kontrast oranını kontrol et. `bg-blue-950` + `text-blue-200` kombinasyonu genelde iyi çalışır.

Sırada Ne Var?

Dört temel bileşenimiz hazır: Button, Input, Card, Alert. Şimdi bunları kullanarak portföy sayfamızı Tailwind'e taşıyacağız (veya yeni bir landing page oluşturacağız).

10 Uygulama: Portföyü Tailwind'e Taşı

Bu bölümde LAB-2 ve LAB-3'te saf CSS ile yazdığın portföy sayfasının tüm bölümlerini Tailwind class'larına dönüştüreceksin. Alternatif olarak, sıfırdan yeni bir landing page de üretebilirsin.

Uygulama Adımı

Uygulama-10: Portföy Sayfası — Tailwind ile Header ve Navigasyon:

```
<a href="#main-content"
   class="sr-only focus:not-sr-only
          focus:absolute focus:top-0 focus:left-0
          bg-blue-800 text-white p-2 z-50">
  Ana icerige atla
</a>

<header class="sticky top-0 z-40 bg-white
               dark:bg-gray-900 border-b
               border-gray-200 dark:border-gray-700">
  <div class="max-w-6xl mx-auto px-4 py-3
             flex flex-col sm:flex-row
             justify-between items-center gap-3">
    <h1 class="text-xl font-bold text-blue-800
               dark:text-blue-300">
      Ahmet Yilmaz
    </h1>
    <nav aria-label="Ana navigasyon">
      <ul class="flex flex-wrap gap-2">
```

```
        <li><a href="#hakkimda"
          class="px-3 py-1 rounded-md text-gray-700
                 dark:text-gray-300
                 hover:bg-blue-100
                 dark:hover:bg-gray-800
                 transition-colors">
          Hakkimda</a></li>
        <li><a href="#projeler"
          class="px-3 py-1 rounded-md text-gray-700
                 dark:text-gray-300
                 hover:bg-blue-100
                 dark:hover:bg-gray-800
                 transition-colors">
          Projeler</a></li>
        <li><a href="#iletisim"
          class="px-3 py-1 rounded-md text-gray-700
                 dark:text-gray-300
                 hover:bg-blue-100
                 dark:hover:bg-gray-800
                 transition-colors">
          Iletisim</a></li>
      </ul>
    </nav>
  </div>
</header>
```

Hakkımda Bölümü:

```
<section id="hakkimda" class="py-16 px-4">
  <div class="max-w-4xl mx-auto flex flex-col
              md:flex-row items-center
              md:items-start gap-8">
    <figure class="shrink-0">
      <img src="profil.jpg"
           alt="Ahmet Yilmaz vesikalik fotografi"
           class="w-40 h-40 rounded-full
                  object-cover shadow-lg" />
    </figure>
    <div>
      <h2 class="text-3xl font-bold text-gray-900
                 dark:text-white mb-4 text-center
                 md:text-left">
        Hakkimda
      </h2>
      <p class="text-gray-600 dark:text-gray-400
                mb-4 leading-relaxed">
        Frontend gelistirici olarak modern web
        teknolojileriyle kullanici dostu
        arayuzler olusturuyorum.
      </p>
      <ul class="flex flex-wrap gap-2">
```

```
        <li class="bg-blue-800 text-white px-3
                   py-1 rounded-full text-sm">
          React</li>
        <li class="bg-blue-800 text-white px-3
                   py-1 rounded-full text-sm">
          TypeScript</li>
        <li class="bg-blue-800 text-white px-3
                   py-1 rounded-full text-sm">
          Tailwind</li>
      </ul>
    </div>
  </div>
</section>
```

Projelerim Bölümü (Card component'ini kullan):

```
<section id="projeler" class="py-16 px-4
                              bg-gray-50
                              dark:bg-gray-900">
  <div class="max-w-6xl mx-auto">
    <h2 class="text-3xl font-bold text-center
               text-gray-900 dark:text-white mb-10">
      Projelerim
    </h2>
    <div class="grid grid-cols-1 sm:grid-cols-2
                lg:grid-cols-3 gap-6">
      <Card variant="elevated" title="E-Ticaret"
            image="proje1.jpg"
            imageAlt="E-Ticaret anasayfa gorunumu">
        React ve Node.js ile tam kapsamli uygulama.
      </Card>
      <!-- Diger kartlar... -->
    </div>
  </div>
</section>
```

İletişim Formu (Input ve Button component'lerini kullan):

```
<section id="iletisim" class="py-16 px-4">
  <div class="max-w-lg mx-auto">
    <h2 class="text-3xl font-bold text-center
               text-gray-900 dark:text-white mb-8">
      Iletisim
    </h2>
    <form class="space-y-4">
      <Input id="name" label="Ad Soyad"
             required />
      <Input id="email" label="E-posta"
             type="email" required />
      <div class="space-y-1">
        <label htmlFor="message"
          class="block text-sm font-medium
```

```
                  text-gray-700
                  dark:text-gray-300">
            Mesajiniz
          </label>
          <textarea id="message" rows="5" required
            class="w-full px-3 py-2 rounded-lg
                  border border-gray-300
                  focus:ring-2 focus:ring-blue-500
                  focus:outline-none
                  dark:bg-gray-800
                  dark:text-gray-100
                  dark:border-gray-600">
          </textarea>
        </div>
        <Button variant="primary" size="lg"
                type="submit">
          Gonder
        </Button>
      </form>
    </div>
  </section>
```

Footer:

```
<footer class="bg-gray-100 dark:bg-gray-900
               border-t border-gray-200
               dark:border-gray-700
               text-center py-6 px-4
               text-gray-500 dark:text-gray-400
               text-sm">
  <p>&copy; 2025 Ahmet Yilmaz.
     Tum haklari saklidir.</p>
</footer>
```

Sırada Ne Var?

Portföyü Tailwind'e taşıdık. Şimdi tüm bileşenlerimizi tek bir sayfada sergileyen **UI Kit** sayfasını oluşturacağız.

11 UI Kit Sayfası Oluşturma

UI Kit, component kütüphaneni **görsel olarak belgeleyen** bir sayfadır. Her bileşenin tüm varyantlarını tek bir yerde gösterir — hem sen hem ekip arkadaşların hangi bileşenlerin mevcut olduğunu hızlıca görür.

Uygulama Adımı

Uygulama-11: UI Kit Sayfası — En Az 8 Varyant
`src/pages/UIKit.jsx` (veya `src/UIKit.jsx`) dosyası oluştur:

```
import Button from '../components/Button';
import Input from '../components/Input';
import Card from '../components/Card';
import Alert from '../components/Alert';

export default function UIKit() {
  return (
    <div className="min-h-screen bg-white
      dark:bg-gray-950 p-8 space-y-12">

      <h1 className="text-4xl font-bold
        text-gray-900 dark:text-white">
        UI Kit
      </h1>

      {/* --- BUTTONS --- */}
      <section className="space-y-4">
        <h2 className="text-2xl font-semibold
          border-b pb-2">Buttons</h2>
        {/* Varyant 1: Primary */}
        <div className="flex flex-wrap gap-3">
          <Button variant="primary">Primary</Button>
          <Button variant="secondary">
            Secondary</Button>
          <Button variant="danger">Danger</Button>
          <Button variant="ghost">Ghost</Button>
        </div>
        {/* Varyant 2: Boyutlar */}
        <div className="flex flex-wrap
                        items-end gap-3">
          <Button size="sm">Small</Button>
          <Button size="md">Medium</Button>
          <Button size="lg">Large</Button>
        </div>
      </section>

      {/* --- INPUTS --- */}
      <section className="space-y-4 max-w-md">
        <h2 className="text-2xl font-semibold
          border-b pb-2">Inputs</h2>
        {/* Varyant 3: Normal */}
        <Input id="ui-name" label="Normal Input"
          placeholder="Bir sey yazin..." />
        {/* Varyant 4: Hatali */}
        <Input id="ui-err" label="Hatali Input"
          error="Bu alan zorunludur" />
        {/* Varyant 5: Help text */}
        <Input id="ui-help" label="Help Text"
          type="email"
          helpText="E-posta adresinizi girin" />
```

```
        {/* Varyant 6: Disabled */}
        <Input id="ui-dis" label="Disabled"
          disabled value="Duzenlenemez" />
      </section>

      {/* --- CARDS --- */}
      <section className="space-y-4">
        <h2 className="text-2xl font-semibold
          border-b pb-2">Cards</h2>
        <div className="grid grid-cols-1
          sm:grid-cols-2 lg:grid-cols-3 gap-6">
          {/* Varyant 7: Elevated */}
          <Card variant="elevated"
            title="Elevated Card">
            Golge ile yukseltiilmis kart.
          </Card>
          {/* Varyant 8: Outlined */}
          <Card variant="outlined"
            title="Outlined Card">
            Cerceveli kart.
          </Card>
          <Card variant="filled"
            title="Filled Card">
            Dolgulu arka plan.
          </Card>
        </div>
      </section>

      {/* --- ALERTS --- */}
      <section className="space-y-4 max-w-xl">
        <h2 className="text-2xl font-semibold
          border-b pb-2">Alerts</h2>
        <Alert variant="info" title="Bilgi">
          Bilgilendirme mesaji.</Alert>
        <Alert variant="success" title="Basarili">
          Islem tamamlandi.</Alert>
        <Alert variant="warning" title="Uyari">
          Dikkat edilmesi gereken durum.</Alert>
        <Alert variant="error" title="Hata"
          dismissible>
          Bir hata olustu.</Alert>
      </section>

    </div>
  );
}
```

Bu sayfada en az **8 farklı component varyantı** sergilenmelidir. Yukarıdaki örnekte 8+ varyant mevcut: Primary/Secondary/Danger/Ghost butonlar, Normal/Hatalı/Help/Disabled inputlar, Elevated/Outlined/Filled kartlar, Info/Success/Warnin-

g/Error alert'ler.

Dikkat – Sık Yapılan Hatalar

- UI Kit sayfası sadece teslim için değil, **geliştirme sürecinde** de faydalıdır. Yeni bir bileşen eklediğinde veya var olanı değiştirdiğinde tüm varyantları tek bakışta kontrol edersin.
- Karanlık temada tüm varyantların doğru göründüğünü kontrol et. UI Kit sayfasında dark mode toggle butonunu kullanarak iki temayı karşılaştır.
- Her varyantın altına kısa bir açıklama (ne zaman kullanılacağı) eklersen dokümantasyon kalitesi artar.

Sırada Ne Var?

UI Kit sayfası hazır. Şimdi tüm çalışmayı Git ile kayıt altına alıp teslim edeceğiz.

12 Git İş Akışı ve Teslim

Uygulama Adımı

Uygulama-12: Git ile Kaydet ve GitHub'a Yükle

1. Yeni branch aç:

```
git checkout -b feature/tailwind-ui-kit
```

2. Adım adım commit at:

```
git add vite.config.js package.json src/index.css
git commit -m "feat: install and configure Tailwind CSS v4"

git add src/components/Button.jsx
git commit -m "feat: add Button component with variants"

git add src/components/Input.jsx
git commit -m "feat: add Input component with error state"

git add src/components/Card.jsx src/components/Alert.jsx
git commit -m "feat: add Card and Alert components"

git add src/
git commit -m "feat: migrate portfolio to Tailwind CSS"

git add src/pages/UIKit.jsx
git commit -m "feat: add UI Kit page with 8+ variants"

git add src/
git commit -m "feat: add dark mode support"
```

3. GitHub'a push et:

```
git push -u origin feature/tailwind-ui-kit
```

13 Teslim Kontrol Listesi ve Değerlendirme Rubriği

Kontrol Listesi

Teslim öncesi kontrol listesi:

- ☐ Tailwind CSS v4 kurulu ve çalışıyor mu? (`npm run dev` ile hata yok)
- ☐ `@theme` ile özel renk/font tanımlanmış mı?
- ☐ Responsive prefix'ler (`sm:`, `md:`, `lg:`) kullanılmış mı?
- ☐ Dark mode (`dark:`) desteği eklenmiş mi? Toggle butonu çalışıyor mu?
- ☐ Button component en az 4 renk varyantına sahip mi?
- ☐ Input component hata durumunu (`error`) destekliyor mu?
- ☐ Card component en az 2 stil varyantına sahip mi?
- ☐ Alert component en az 4 renk varyantına sahip mi?
- ☐ UI Kit sayfasında en az 8 component varyantı sergilenmiş mi?
- ☐ Portföy sayfası (veya landing page) Tailwind ile stillenmiş mi?
- ☐ Erişilebilirlik korunmuş mu? (`focus:ring`, `aria-label`, skip link, `role="alert"`)

- ☐ En az 3 anlamlı commit atılmış ve branch kullanılmış mı?
- ☐ GitHub'a push edilmiş mi?

13.1 Değerlendirme Rubriği

Kriter	Puan	Açıklama
Tailwind kurulum ve yapılandırma	10	v4 entegrasyonu, @theme
Utility-first class kullanımı	10	Doğru ve tutarlı class seçimi
Responsive prefix kullanımı	10	sm:/md:/lg: ile 3 breakpoint
Dark mode desteği	10	dark: prefix, toggle butonu
Button component (4+ varyant)	10	Renk + boyut varyantları
Input component (hata/help/disabled)	10	Erişilebilir form alanı
Card component (2+ varyant)	10	elevated/outlined/filled
Alert component (4 renk)	10	info/success/warning/error
UI Kit sayfası (8+ varyant)	10	Tüm bileşenler tek sayfada
Git iş akışı (branch + commit)	10	En az 3 anlamlı commit
Toplam	**100**	

14 Kendini Test Et

Kendini Test Et

Aşağıdaki sorulara kendi cümlelerinle 2–3 satır cevap yazabiliyor musun? Yazamıyorsan ilgili bölümü tekrar oku.

1. **Utility-first CSS nedir? Geleneksel (semantic) CSS'ten farkı nedir?**
2. **Tailwind'in "inline style'dan farkı ne" sorusuna nasıl cevap verirsin?**
3. **Tailwind responsive prefix'leri mobile-first mıdır? `md:flex-row` ne anlama gelir?**
4. **Tailwind'in renk skalasında `blue-50` ve `blue-950` arasındaki fark nedir?**
5. **Dark mode için `dark:` prefix'i nasıl çalışır? Hangi iki strateji vardır?**
6. **Component yaklaşımı neden gereklidir? 10 yerde aynı buton class'larını kopyalamak neden sorunludur?**
7. **React component'i ile `@apply` arasındaki fark nedir? Hangisi önerilir?**
8. **Button component'inde `focus:ring` neden önemlidir? Kaldırırsan ne olur?**
9. **Input component'inde `aria-describedby` ne işe yarar?**
10. **UI Kit sayfası neden faydalıdır? Sadece teslim için mi yoksa geliştirme sürecinde de mi?**

15 Ek Kaynaklar ve İleri Okuma

- **Tailwind CSS Dokümantasyonu:** https://tailwindcss.com/docs — Resmi ve kapsamlı referans. Her class detaylı örneklerle açıklanır.
- **Tailwind Play:** https://play.tailwindcss.com — Tarayıcıda canlı Tailwind CSS denemesi. Kurulum gerektirmez.
- **Tailwind UI:** https://tailwindui.com — Profesyonel, erişilebilir component örnekleri (bazıları ücretsiz).
- **Headless UI:** https://headlessui.com — Tailwind ekibinin geliştirdiği erişilebilir, stillenmemiş React bileşenleri.
- **Heroicons:** https://heroicons.com — Tailwind ekibinin SVG ikon seti. React component olarak kullanılabilir.
- **Tailwind CSS IntelliSense:** VS Code eklentisi — otomatik tamamlama, hover bilgisi, hata tespiti.
- **Prettier Plugin for Tailwind:** https://github.com/tailwindlabs/prettier-plugin-tailwi — Class sıralamasını otomatik düzenler.
- **shadcn/ui:** https://ui.shadcn.com — Tailwind + Radix UI tabanlı kopyala-yapıştır component kütüphanesi. Component tasarımı için ilham kaynağı.